

Large-Scale Network Utility Maximization via GPU-Accelerated Proximal Message Passing

Akshay Sreekumar¹ **Anthony Degleris**² **Ram Rajagopal**¹

¹Department of Electrical Engineering, Stanford University

²Gridmatic Inc.

May 27, 2026



- ▶ **Motivation**
- ▶ Problem Setting
- ▶ Method
- ▶ Results
- ▶ Future Work

Resource Allocation in Large Networks

Many domains

- ▶ Communication networks
- ▶ Sensor networks
- ▶ Power systems
- ▶ Transit systems

Common requirements

- ▶ Shared capacity limits
- ▶ Allocate efficiently
- ▶ Preserve fairness
- ▶ Adapt to congestion or degradation

Outline

- ▶ Motivation
- ▶ **Problem Setting**
- ▶ Method
- ▶ Results
- ▶ Future Work

Large-Scale Network Utility Maximization

- ▶ **Network utility maximization (NUM)** is a canonical network resource allocation problem¹
- ▶ Streams compete for capacity on shared links; the goal is to maximize aggregate utility

$$\begin{aligned} \underset{x}{\text{maximize}} \quad & \sum_{j=1}^n U_j(x_j) \\ \text{s.t.} \quad & Rx + s = c, \\ & x \geq 0, s \geq 0 \end{aligned} \tag{1}$$

- ▶ $x \in \mathbb{R}^n$: stream rates, $s \in \mathbb{R}^m$: slacks, $c_i > 0$: link capacities
- ▶ $R \in \mathbb{R}^{m \times n}$: link-stream matrix, U_j are concave utilities

$$R_{ij} = \begin{cases} 1 & \text{if stream } j \text{ uses link } i, \\ 0 & \text{otherwise} \end{cases}$$

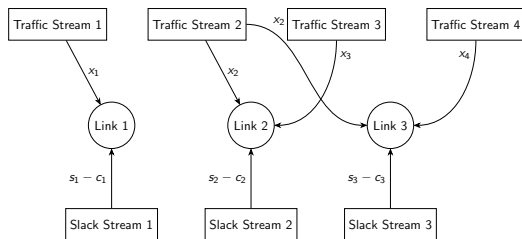
NUM: Bipartite Graph View

$$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} + \begin{pmatrix} s_1 \\ s_2 \\ s_3 \end{pmatrix} - \begin{pmatrix} c_1 \\ c_2 \\ c_3 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$$

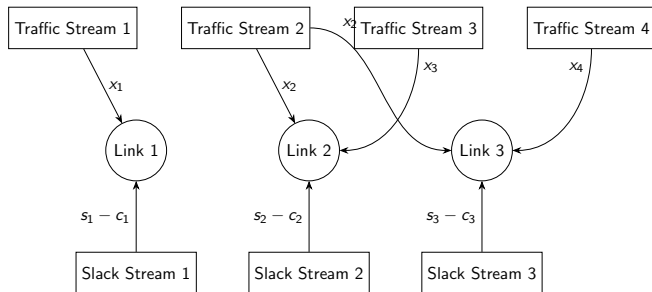
- NUM is represented as a **bipartite graph** with:

- streams** $\sigma \in \mathcal{S}$
- links** $l \in \mathcal{L}$
- terminals** $j \in \mathcal{J}$ connecting them

- Terminal flows are collected in $p \in \mathbb{R}^{|\mathcal{J}|}$
- Use $p_\sigma \in \mathbb{R}^{|\sigma|}$ and $p_l \in \mathbb{R}^{|\mathcal{I}|}$ for flows incident to stream σ and link l



NUM: Bipartite Graph View II



- ▶ Convex indicator $\delta_C(x)$: $\delta_C(x) = 0$ if $x \in C$, and $+\infty$ otherwise
- ▶ Streams are nodes with costs for producing or consuming flow
 - ▶ e.g. $f_\sigma(p_\sigma) = \underset{x_\sigma \geq 0}{\text{minimize}} \left(-\log x_\sigma + \delta_{\{0\}}(p_\sigma - \mathbf{1}_{|\sigma|} x_\sigma) \right)$
- ▶ Links are nodes with flow balance constraints
 - ▶ e.g. $g_l(p_l) = \delta_{\{0\}}(\mathbf{1}^\top p_l)$

Outline

- ▶ Motivation
- ▶ Problem Setting
- ▶ **Method**
- ▶ Results
- ▶ Future Work

NUM: ADMM Form and Proximal Message Passing

$$\begin{array}{ll} \underset{p, z}{\text{minimize}} & \sum_{\sigma \in \mathcal{S}} f_{\sigma}(p_{\sigma}) + \sum_{l \in \mathcal{L}} g_l(z_l) \\ \text{subject to} & p = z \end{array}$$

- ▶ Re-write with p for **stream-side** variables and z as **link-side** copy
- ▶ Apply ADMM to derive *GPU-accelerated proximal message passing*
 - ▶ Variant of ADMM using only sparse matrix–vector products
 - ▶ Alternates between (i) **stream-side proximal updates**
 - ▶ and (ii) **price updates**

NUM: From ADMM to Message Passing

ADMM updates

$$p_{\sigma}^{k+1} := \arg \min_{p_{\sigma}} \left(f_{\sigma}(p_{\sigma}) + \frac{\rho}{2} \|p_{\sigma} - z_{\sigma}^k + u_{\sigma}^k\|_2^2 \right)$$

$$z_l^{k+1} := \arg \min_{z_l} \left(g_l(z_l) + \frac{\rho}{2} \|z_l - p_l^{k+1} - u_l^k\|_2^2 \right)$$

$$u_l^{k+1} := u_l^k + p_l^{k+1} - z_l^{k+1}$$

- ▶ u_{σ} and u_l are the scaled dual variables for streams and links, respectively
- ▶ $\bar{p} \in \mathbb{R}^{|\mathcal{J}|}$ is a vector where each terminal entry is the average of all terminal flows incident to the terminal's corresponding link

$$\boxed{\begin{aligned} p_{\sigma}^{k+1} &= \text{prox}_{f_{\sigma}, \rho}(p_{\sigma}^k - \bar{p}_{\sigma}^k - u_{\sigma}^k), & \sigma \in \mathcal{S} \\ u_l^{k+1} &= u_l^k + \bar{p}_l^{k+1}, & l \in \mathcal{L} \end{aligned}}$$

Why “message passing”? Streams send flows to links; links send prices

PMP exposes two operations that map naturally to GPUs

1. Stream-side proximal updates

- ▶ Streams with the same utility type and route length are stored together
- ▶ Log, linear, and slack updates have closed-form proximal operators
- ▶ Each stream group is updated by batched PyTorch tensor operations

2. Link-side averaging

- ▶ Each terminal flow belongs to exactly one link
- ▶ For each link, compute the average incident flow:

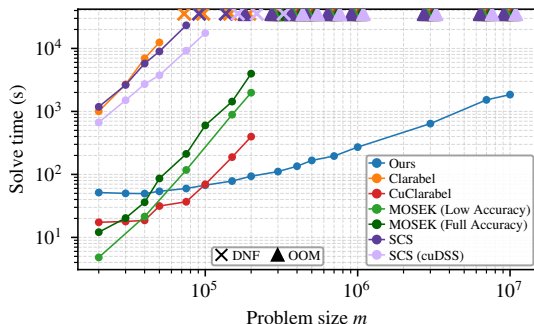
$$\bar{p}_l = \frac{1}{\#\{\text{terminals on } l\}} \sum_{\text{terminal } j \text{ incident to } l} p_j$$

- ▶ Implemented efficiently using GPU scatter kernel

Outline

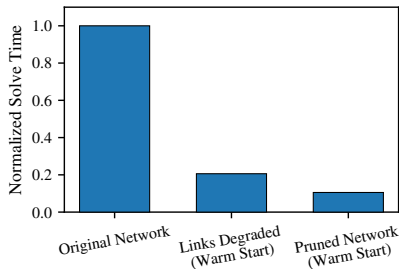
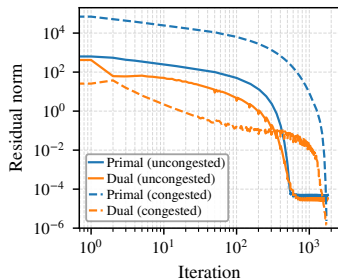
- ▶ Motivation
- ▶ Problem Setting
- ▶ Method
- ▶ **Results**
- ▶ Future Work

Scaling



- Scales to millions of links and streams on a single GPU (A100)
- Up to $4\times$ faster than CuClarabel and $20\times$ faster than MOSEK
- For $m \geq 5 \times 10^5$, it was the only method that solved the instances without running out of memory

Robustness: Congestion and Link Degradation



- Congest 0.1% of links by connecting to $\sim 10\%$ of streams; moderate convergence with few hundred extra iterations
- Warm-starting from previous solution gives $\sim 5\times$ speedup after random capacity degradation
- Pruning failed links and affected streams, plus warm-starting, gives $\sim 10\times$ speedup

Outline

- ▶ Motivation
- ▶ Problem Setting
- ▶ Method
- ▶ Results
- ▶ **Future Work**

- ▶ Exploit **differentiability** for link capacity planning via gradient based methods
 - ▶ How could we optimally plan link capacities knowing that you will use NUM to allocate afterwards?
- ▶ Apply framework to new application domains
 - ▶ Power control in AI datacenter resource allocation
- ▶ Proximal message passing can be used more generally to solve conic optimization problems
 - ▶ Extended framework to solve general LPs, QPs, SOCPs
 - ▶ But needs far more work to be competitive with other GPU accelerated conic solvers



Paper

Check out the full paper for all the details!



Code

Our Python library for differentiable network optimization (includes NUM, DC-OPF, can be extended to other problems)